SYMBIOSIS INSTITUTE OF TECHNOLOGY,HYDERABAD
Modallaguda (V), Nandigama (M), Rangareddy Dist, Hyderabad, Telangana, India, Pin Code : 509217

# Subject / Course – Data Structures and Algorithms

## **Topic –** Report on Music lab Playlist Manager Using a Doubly Linked List

CA-2 Experiential Learning Report

BTech CSE (AI & ML) | Second Year

**Student name:** Rajveer Vadnal

**PRN / Roll number:** 26070725501

**Programme:** BTech CSE (AI & ML)

**Semester / Section:** 3rd Semester

**Assessment:** Lab CA-2 — Experiential Learning Task

**Date of submission:** 22-10-2026

**Language and tools:** C11, GCC, Make, HTML, CSS and JavaScript

**Github Repo -** https://github.com/Rajveerx11/DSAProject

### Project focus

A local music playlist application that uses C pointers to manage song order and a browser view to explain the linked-list operations.

Report version 5 | Verification date 14 October 2026

Source version f162a82a | Individual demonstration and viva 10 minutes

Complete runnable source and supporting evidence accompany this report.

# Index


| Section | Topic | Page |
|---|---|---|
| 1 | Abstract and Summary | 3 |
| 2 | Problem Statement and Real World Relevance | 4 |
| 3 | Objectives | 5 |
| 4 | Data Structure and Algorithm Selected | 6 |
| 5 | System Design and Approach | 7 |
| 6 | Implementation Details | 10 |
| 7 | Test Cases and Results | 11 |
| 8 | Execution Evidence | 13 |
| 9 | Analysis of Time and Space Complexity | 16 |
| 10 | Challenges and Learning Outcomes | 17 |
| 11 | Conclusion | 18 |
| 12 | References | 19 |
| A | Source Code and Companion Package | 20 |
| A1 | Core Data Structures | 21 |
| A2 | Event Recording and Metadata Validation | 22 |
| A3 | Append Search and Deletion | 23 |
| A4 | Selection Navigation and Reversal | 24 |
| A5 | Concatenation and Memory Cleanup | 25 |

# 1. Abstract and Summary

Musiclab is an interactive playlist manager that demonstrates the practical implementation of a doubly linked list in C. It maintains two playlists, with each song represented by a dynamically allocated node containing a unique ID, title, artist, and pointers to its previous and next neighbours. Each playlist also stores head, tail, current and size fields to support insertion, bidirectional navigation, selection, deletion, reversal and concatenation. The C backend manages all playlist operations and memory ownership, while the browser provides a Demo view with playback controls and a Simulation view that visualises recorded pointer changes.

The project shows how individual pointer assignments produce meaningful changes in playlist behaviour. Deleting the current song updates neighbouring links and selects the next available song, the previous song, or NULL when the playlist becomes empty. Reversal exchanges every node's previous and next pointers and swaps the list endpoints while retaining the current song. Concatenation transfers existing nodes from one playlist to another without copying them and clears the source playlist's ownership references. The Simulation view presents events recorded by the C implementation, allowing users to examine allocations, search visits, pointer assignments and deallocations.

This report explains the system architecture, data structures, algorithms, worked examples, execution outputs and complexity analysis. Twelve scenario groups passed, covering normal operations, empty and single-node playlists, invalid IDs, reversal and merge behaviour. Native, API, trace and sanitizer checks also passed. Browser testing identified a replay keyboard-focus issue, which is documented as an interface limitation rather than a failure of the verified linked-list operations.

Maintaining a tail pointer enables constant-time append and concatenation, while searching by ID, deleting by ID and reversing a playlist require linear traversal in the worst case. Although some core operations take O(1) time, copying and serializing playlist state for browser display adds O(N) processing. Musiclab therefore connects classroom DSA concepts with dynamic memory management, boundary conditions, testing and the distinction between algorithmic cost and complete application behaviour.

# 2. Problem Statement and Real World Relevance

The main problem in Musiclab is keeping the playlist valid after its order changes. For example, removing the selected song requires both a new selection and repaired links between the remaining songs. The program must also recognise when the list becomes empty. A correct display alone is insufficient if an old pointer still refers to a deleted node.

Side A and Side B provide a small, concrete setting for these operations. Next and Previous follow neighbouring nodes, Reverse changes their order, and Merge joins Side B to Side A. The Simulation view makes the associated links visible, so each button can be related to a C operation. [2, 3]

**Scope.** C stores song metadata and list links in memory. The browser handles generated previews and optional local audio. The project does not provide persistent storage, online streaming, user accounts or a music catalogue. [2, 3]

## Functional requirements

The manager must add a song at the end, select a song by ID, move to an adjacent song, remove a selected record, reverse a playlist and concatenate the two lists. Every successful change must leave the endpoints, node count and selection consistent. Invalid requests must return a useful error without changing the existing playlist. [2, 3]

## A practical usage example

A listener can keep a short practice set in Side A and another group of tracks in Side B. After removing an unwanted song, the listener can reverse the practice order or attach Side B to the end. Musiclab models the ordering and navigation needed for this workflow. The sample audio and browser controls make the operations easy to demonstrate, while the node view explains their internal effect.

## Scope and constraints

The project is a local DSA demonstration. It stores up to 200 song records across both playlists during a server run. A record holds metadata and links; it does not store an audio file in C. Closing the server loses the playlist state. These limits keep the implementation focused on pointer operations, input validation and memory management. A persistent music library would need additional storage and file-management features. [3]

# 3. Objectives

## Objective 1 Implement the playlist operations

Use a doubly linked list in C to support append, select, delete, Next, Previous, reverse and concatenate. The implementation must update both link directions and maintain head, tail, current and size. It must also allocate and release nodes safely, including when the first song is added or the last song is removed.

## Objective 2 Explain how the pointers change

Connect the visible playlist with the underlying C operations. The node diagram identifies each field, the pseudocode explains the assignment order, and the Simulation view replays recorded changes. A worked deletion and a reversal dry run show how individual assignments produce the final order. The explanation must distinguish a song's ID from its position or memory address.

## Objective 3 Evaluate correctness and efficiency

Compare expected and actual outputs for normal operations, empty and single-node lists, and invalid input. Evaluate time and space requirements using the number of stored nodes. In particular, separate the search needed for deletion by ID from the constant-time repair of a node that has already been found.

## Evidence used to assess the objectives

Sections 4–6 connect the node design with the implemented functions. Sections 7 and 8 provide API results and repository execution screenshots. Section 9 explains the costs of the C operations and the additional work involved in recording and displaying their results. The appendix provides the core implementation for inspection during the viva.

The objectives concern the implemented DSA behaviour. They do not claim a production music service, permanent storage or fully resolved interface testing. The reported replay-focus issue is retained as a limitation, so successful list tests are not presented as proof that every interface behaviour is correct.

# 4. Data Structure and Algorithm Selected

Musiclab stores each song in a SongNode with an ID, two 128-byte text arrays and prev and next links. Playlist keeps head, tail, current and size. Each field has a separate role: current can remain on the same song while Reverse exchanges head and tail. Both list ends are NULL. [2]

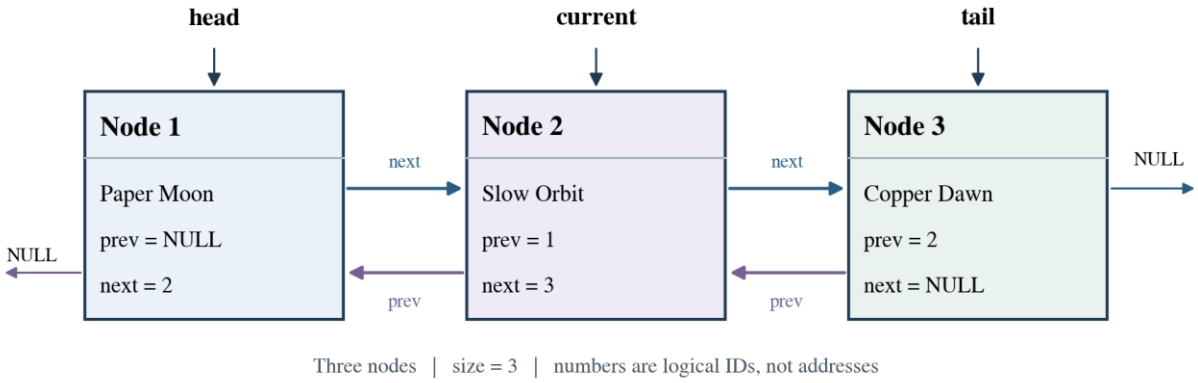



**Figure 1.** A three-song playlist. current identifies the selected node; NULL marks each endpoint.

| Field | Role |
|---|---|
| head / tail | Locate the first and last nodes without traversal. |
| current | Identify the selected song independently of its position. |
| prev / next | Connect each node to its immediate neighbours. |
| size | Record the number of nodes owned by the playlist. |

Search follows next until it finds a matching ID; Reverse visits every node. Append uses tail directly, while Merge joins the stored endpoints. A doubly linked list therefore helps some operations without making every operation constant-time. [2]

| Choice | Benefit for this project | Trade-off |
|---|---|---|
| Doubly linked list | Direct Next/Previous; local link changes; endpoint concatenation. | Two pointers per node; lookup by ID still needs traversal. |
| Singly linked list | One link per node; append is efficient when tail is kept. | Previous from current normally requires a scan. |
| Array | Direct indexing; Next/Previous can use an index. | Middle deletion shifts records; joining storage may require copying. |

**Correctness conditions.** In a non-empty list, head->prev and tail->next are NULL, neighbouring links agree, size matches the node count, and current belongs to the list. An empty list has NULL endpoints/current and size zero. Tests check these conditions after completed operations. [1, 2, 4]

The prev link gives direct access to a known node's predecessor during deletion. This requires an extra pointer per node and careful updates in both directions. Finding an ID still needs traversal. [1, 2]

# 5. System Design and Approach

A click in the browser becomes a request to the local C server. The API checks the request before calling a playlist function. Its response contains the completed list state and the events recorded during the operation. JavaScript uses these events to explain what the C code has already done. [3]

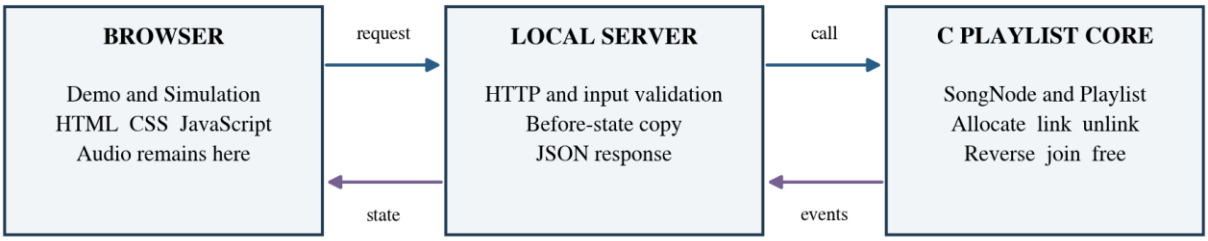



**Figure 2.** Separation of interface, request handling and C linked-list operations.

| Stage | What happens |
|---|---|
| Input | Choose a playlist and operation; enter a title/artist or select a song ID. |
| Validation | Check action, playlist, ID, metadata and capacity. Reject invalid input without partially changing the state. |
| Processing | Save the before state, perform the C operation, and record allocation, visits, pointer writes and frees. |
| Output | Return the committed playlists and event history as JSON; update the interface and explain the operation. |

## How the two views support the project

Demo provides player controls and playlist editing. Simulation adds a visual explanation of the same C operations. The optional guided tour performs eight actions: Reset, Add, Next, Previous, Select, Delete, Reverse and Merge.

The C code reports changes through an observer callback. The collected events form a trace: an ordered record of visits, assignments and memory operations. The API writes this information as JSON for the browser. This makes it possible to inspect an assignment such as tail.next changing from NULL to a new node.

A replay pauses between assignments, so a frame can show a link that has been changed before its partner. Such frames describe work in progress; the completed list must satisfy the correctness conditions. Reset replaces the sample lists and returns an empty trace. [3]

For example, Delete sends an ID to the API. After validation, C locates the node, repairs the links and releases it. The response then supplies the remaining songs and the event sequence used by Simulation. [2, 3]

The following pseudocode focuses on the list operations. It omits event-recording calls so that the pointer logic is easier to follow. The complete C implementation appears in Appendix A.

### Append a song

```
APPEND(list, id, title, artist)
  Validate metadata, ID and capacity; reject invalid input.
  Allocate a node; if allocation fails, leave the list unchanged.
  Store the ID and metadata; set node.next = NULL.
  node.prev = list.tail
  if list.tail exists: list.tail.next = node
  else: list.head = node
  list.tail = node
  if list.current is NULL: list.current = node
  list.size = list.size + 1
```

For the first song, head, tail and current all point to the new node. For later songs, current stays selected and tail moves to the new node. The allocation uses calloc; a failed allocation returns without adding a record. [2]

### Delete a song by ID

```
DELETE(list, id)
  node = FIND(list, id)
  if node is NULL: report not found and stop
  if node == list.current:
    list.current = node.next if present, otherwise node.prev
  if node.prev exists: node.prev.next = node.next
  else: list.head = node.next
  if node.next exists: node.next.prev = node.prev
  else: list.tail = node.prev
  list.size = list.size - 1
  free(node)
```

Deleting an endpoint needs a separate update because one neighbour is missing. Deleting the only node leaves head, tail and current NULL. The selected node is replaced before free is called, so current does not retain an address that the list has released. [2]

### Worked deletion example

Start with A = [1, 2, 3] and current = 2. Finding ID 2 visits nodes 1 and 2. Set current to 3, set node 1.next to 3, and set node 3.prev to 1. Decrease size from 3 to 2, then free node 2. The final order is [1, 3], with head = 1 and tail = 3.

The search is O(n) in the worst case. Once the node has been found, repairing its links and freeing it take O(1) time.

The same deletion rules cover the head, tail and only node. Testing those cases is necessary because one or both neighbour pointers may be NULL. [2, 7]

### Reverse the playlist

```
REVERSE(list)
  node = list.head
  while node is not NULL:
```

```
    savedNext = node.next
    node.next = node.prev
    node.prev = savedNext
    node = savedNext
  swap(list.head, list.tail)
  Keep list.current and list.size unchanged.
```

savedNext keeps the route to the next unprocessed node. Without it, swapping the links would also change the route used by the loop. The table below follows those assignments for the three sample nodes; no replacement nodes are allocated. [2]

| Node | Saved next | New prev | New next |
|---|---|---|---|
| 1 | 2 | 2 | NULL |
| 2 | 3 | 3 | 1 |
| 3 | NULL | NULL | 2 |

**Table 1.** Dry run for [1, 2, 3]. After the loop, head = 3 and tail = 1. If current was node 2, it remains node 2.

## Concatenate B into A

```
CONCATENATE(A, B)
  Reject A == B or a combined size above the limit.
  if B is empty: stop without changing either list
  if A is empty:
    A.head = B.head; A.current = B.current
  else:
    A.tail.next = B.head; B.head.prev = A.tail
  A.tail = B.tail; A.size = A.size + B.size
  B.head = NULL; B.tail = NULL; B.current = NULL; B.size = 0
```

After the join, A owns the nodes that previously belonged to B. Clearing B prevents later cleanup from freeing those nodes twice. A keeps its selection if it already contains songs; when A is empty, it takes B.current. Test T11 checks this second case. [2, 7]

The Merge button preserves the order within each list. For example, joining [4, 5] to [6, 3, 1] gives [6, 3, 1, 4, 5]. This is concatenation: it needs a fixed set of link and counter updates and has O(1) core cost. [2, 7]

After reversal, traversing from the new head with next must visit the old order backwards. Traversing from the new tail with prev must recover the original order. This gives a direct way to check that both sets of links were exchanged correctly. [2, 4]

# 6. Implementation Details

The project uses C11 with warning-enabled compilation. GCC (or another compatible C11 compiler) and Make build the application. HTML, CSS and JavaScript provide the interface. Linux and Windows through WSL are supported; Node.js is needed only for validation. [3]

| Module | Responsibility |
|---|---|
| src/playlist.h and src/playlist.c | Define nodes; append, find, select, navigate, delete, reverse, concatenate and free. |
| src/api.h and src/api.c | Validate requests, maintain two playlists and return state plus recorded events. |
| src/server.c | Serve the local HTTP interface and allowlisted web files. |
| web/index.html, styles.css and app.js | Display Demo/Simulation, handle browser audio and replay C events. |
| tests/ and web/check.cjs | Check invariants, API behaviour, recorded events and interface behaviour. |

## Important functions in the implementation

Selection starts with playlist_find, which compares IDs while walking from head. playlist_select stores the result in current. By contrast, playlist_move needs only current->next or current->prev. At an endpoint it keeps the selection in place, as checked by the single-song case T9. [2, 7]

The deletion code separates finding a node from unlinking it. playlist_delete accepts an ID, while playlist_delete_node accepts the node itself. This lets the API reuse a completed search. For full cleanup, playlist_clear saves the next address before freeing the current node. [2]

ASSIGN performs a pointer assignment and, when an observer is attached, records its old and new values. With no observer, it only assigns the pointer. Recorded events identify nodes by ID and copy their metadata, allowing the browser to explain a deletion after the node has been freed. [2]

## Input rules and audio behaviour

Title and artist must be non-empty valid UTF-8, with at most 127 bytes each and no control characters. The API allows at most 200 songs across both playlists. IDs remain unique during a server process, including after Reset. The byte limit can allow fewer than 127 characters for multibyte text. Reversing the list changes song positions but preserves their IDs, allowing controls to keep referring to the same record. [2, 3]

Demo generates 12-second sample melodies or plays an optional local audio file. Local files remain in the browser and their associations are temporary. Simulation is silent; C manages metadata and links only.

## Build and run from the project folder

```
make
./build/musiclab --port 8081
# Open http://localhost:8081 in a browser
# Validation: make check; make sanitize
# Optional browser validation: see README.md
```

# 7. Test Cases and Results

Twelve scenario groups ran against the unchanged source version f162a82a on 14 October 2026, using Ubuntu 24.04.3, GCC 13.3.0 and Node.js 24.19.0. Actual outputs come from C API responses. Captured JSON and the reproduction script are included in the supporting package. [7]

**Test sequence A.** T1–T5 run in order from a fresh server: A = [1, 2, 3], B = [4, 5], and A.current = 1. Brackets show node IDs in forward order, not memory addresses.

| Test and input | Expected output | Actual output | Status |
|---|---|---|---|
| T1 Append Evening Loop to A | Append at tail; preserve current. | A [1,2,3,6]; tail 6; current 1. | Pass |
| T2 Next, then Previous | Follow adjacent links without changing order. | Current 2, then 1; A [1,2,3,6]. | Pass |
| T3 Select and delete ID 2 | Reconnect neighbours; current becomes next. | A [1,3,6]; current 3; 1.next = 3 and 3.prev = 1. | Pass |
| T4 Reverse A | Reverse links and endpoints; retain current. | A [6,3,1]; head 6; tail 1; current 3. | Pass |
| T5 Merge B into A | Concatenate; empty B; retain existing IDs. | A [6,3,1,4,5]; B size 0 with NULL endpoints/current. | Pass |

## What these outputs demonstrate

The linked-list result can be checked directly in T3: node 1 points forward to 3, and node 3 points back to 1. In T4, current remains 3 even though the endpoints change. T5 then retains the same node IDs after joining the lists and clears B. These observations connect the outputs to the pointer rules.

The native tests supplement these API results by walking both directions through each list. They check that the counted nodes agree with size and that current belongs to the list. A forward display could look correct even if a prev link were wrong, which is why both directions are checked. [4]

The scenarios run in a stated order so that later outputs can be reproduced. For example, T4 begins with the three nodes remaining after T3; it does not start again from the original sample playlist. [7]

## Representative validation output

```
native: PASS (removals, navigation, reverse, merge,
  UTF-8, bounds, invariants, cleanup, synchronous observers)
api: PASS (604 responses; actions, invariants, limits,
  framing, origins, static allowlist, deadlines, disconnect)
```

This is an excerpt from the fresh make check output. “604 responses” describes the API suite output, not 604 distinct user scenarios. Full logs are included with the source attachment.

**Test sequence B.** T6 begins with a separate fresh server: A = [1, 2, 3] and B = [4, 5]. T6–T12 then run in order. Rejected requests are checked for unchanged state.

| Test and input | Expected output | Actual output | Status |
|---|---|---|---|
| T6 Delete head ID 1 | Move head/current to node 2. | A [2,3]; head/current 2; head.prev NULL. | Pass |

| Test and input | Expected output | Actual output | Status |
|---|---|---|---|
| T7 Delete tail ID 3 | Repair tail; preserve current. | A [2]; head/tail/current 2; tail.next NULL. | Pass |
| T8 Delete ID 2; Next, Previous, Reverse | Empty operations are safe no-ops. | A empty; endpoints/current NULL; three actions HTTP 200. | Pass |
| T9 Append Solo; Next and Previous | Single-node endpoints/current agree. | Only ID 6; head/tail/current 6; no wrapping. | Pass |
| T10 ID 999; ID abc; empty/128-byte title | Reject invalid input; preserve state. | Missing ID: 404. Other inputs: 400. State unchanged. | Pass |
| T11 Clear A; select B ID 5; merge B into A | Empty destination inherits source current. | A [4,5]; current 5; B empty. | Pass |
| T12 Self-merge; then merge empty B | Reject self-merge; empty B is a no-op. | Self-merge 400; empty-source merge 200; lists unchanged. | Pass |

| Verification | Result and evidence |
|---|---|
| Fresh local make check | Passed: native, trace, API, syntax and frontend checks on 14 October 2026. |
| Repository CI run 38446276615 | Passed make check, make sanitize and real-browser flows on 13 October 2026 UTC. Browser log reports 81 replay/layout checks. |
| Fresh local memory/browser runs | LeakSanitizer could not inspect the execution environment; Chromium installation did not complete. No fresh full pass is claimed for these runs. |

**Known issue.** V1 recorded loss of keyboard focus during a replay transition. The archived CI success does not establish that this later observation is resolved. Reproduce and correct the focus behaviour, then rerun the complete browser suite before claiming full UI validation. [8]

The boundary cases exercise different branches: T8 checks no remaining nodes, T9 checks both endpoints on one node, and T11 checks selection transfer into an empty destination. T10 and T12 verify that rejected requests preserve state. [7]

# 8. Execution Evidence

The following are genuine screenshots from this project's browser tests, retrieved from the successful GitHub Actions run 38446276615 for source version f162a82a. They were captured on 13 October 2026 UTC. They are execution evidence, separate from the explanatory diagrams in Sections 4 and 5. [5]

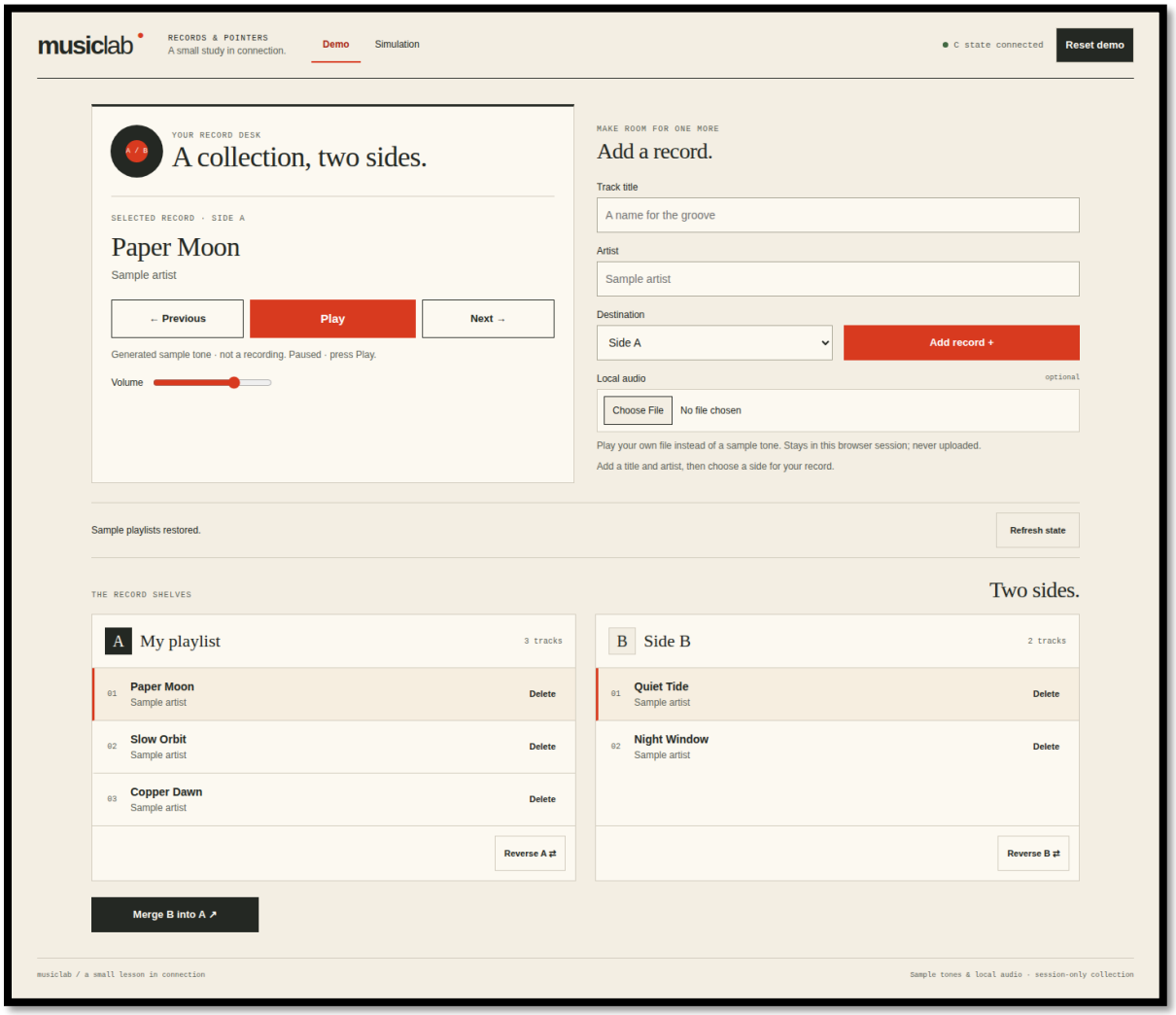



**Figure 3.** Demo view with the initial three-song Side A and two-song Side B. The interface provides add, navigation, delete, reverse, merge and browser audio controls.

The initial records are sample metadata. Generated previews are sample melodies rather than recordings of the displayed song titles. This screenshot shows the application interface and populated playlists; it does not by itself prove audio output. The browser test log records the audio checks.

## Guided run before the append action

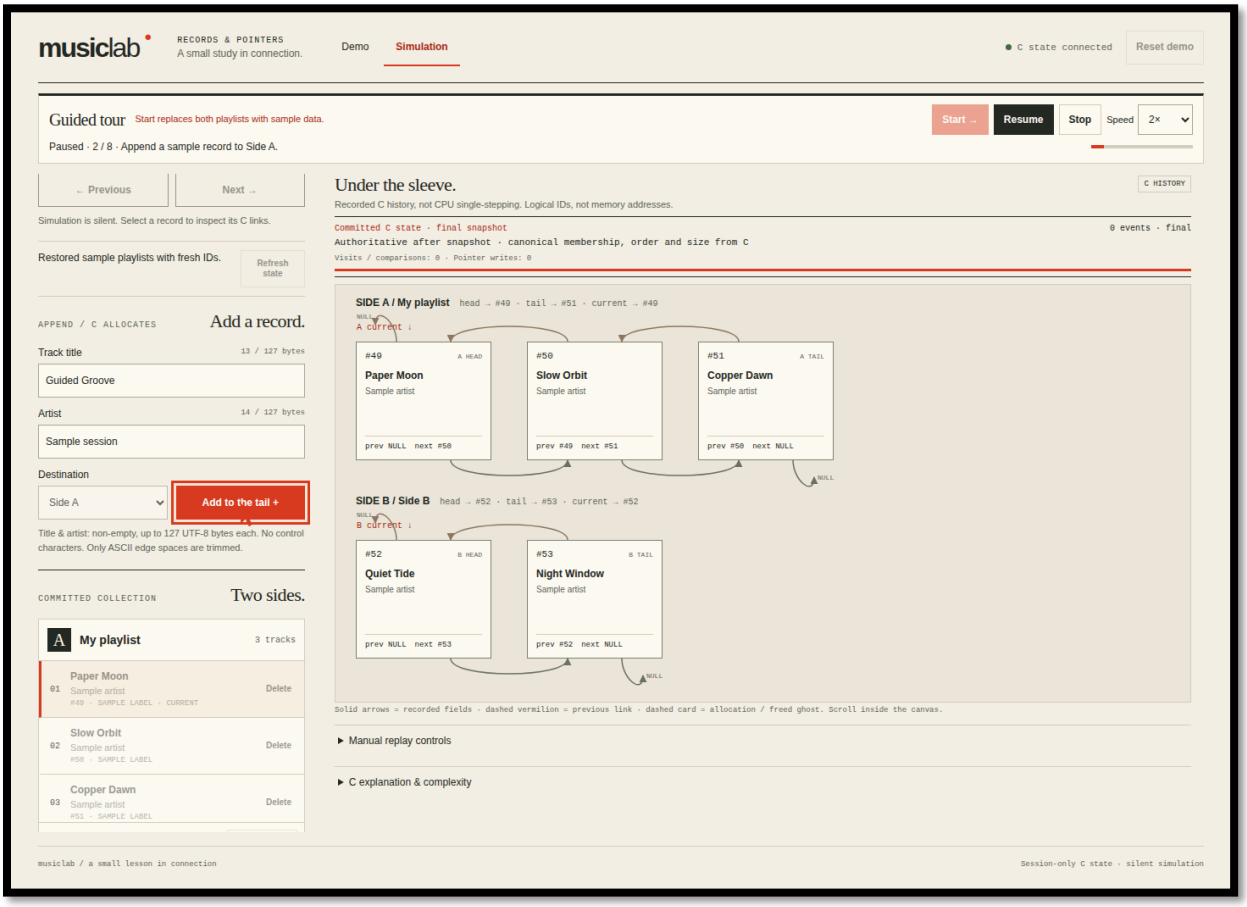



**Figure 4.** The guided tour is paused at the append step. C holds A = [49, 50, 51] and B = [52, 53]; current in A is 49. The title and artist inputs are filled, while the pending new record has not yet been appended.

The highlighted Add control is ready to append a song, but the node display still shows the original three songs in A. Labels identify the endpoints and selected song, while the arrows show the two link directions. The numbers identify nodes within this server process; they are not memory addresses.

The following screenshot is from the same guided sequence after all eight actions complete. IDs differ from the fresh API tests in Section 7 because this browser test process had already created and reset playlists. Reset allocates fresh IDs; it does not restart the ID sequence.

## Reading the node display

A node card combines metadata with the two link destinations. Following next from head gives the displayed order; following prev from tail should give the reverse order. The separate current label shows the selected node. This allows selection to be checked independently of the list's endpoints. [3, 5]

## Guided run after reversal and concatenation

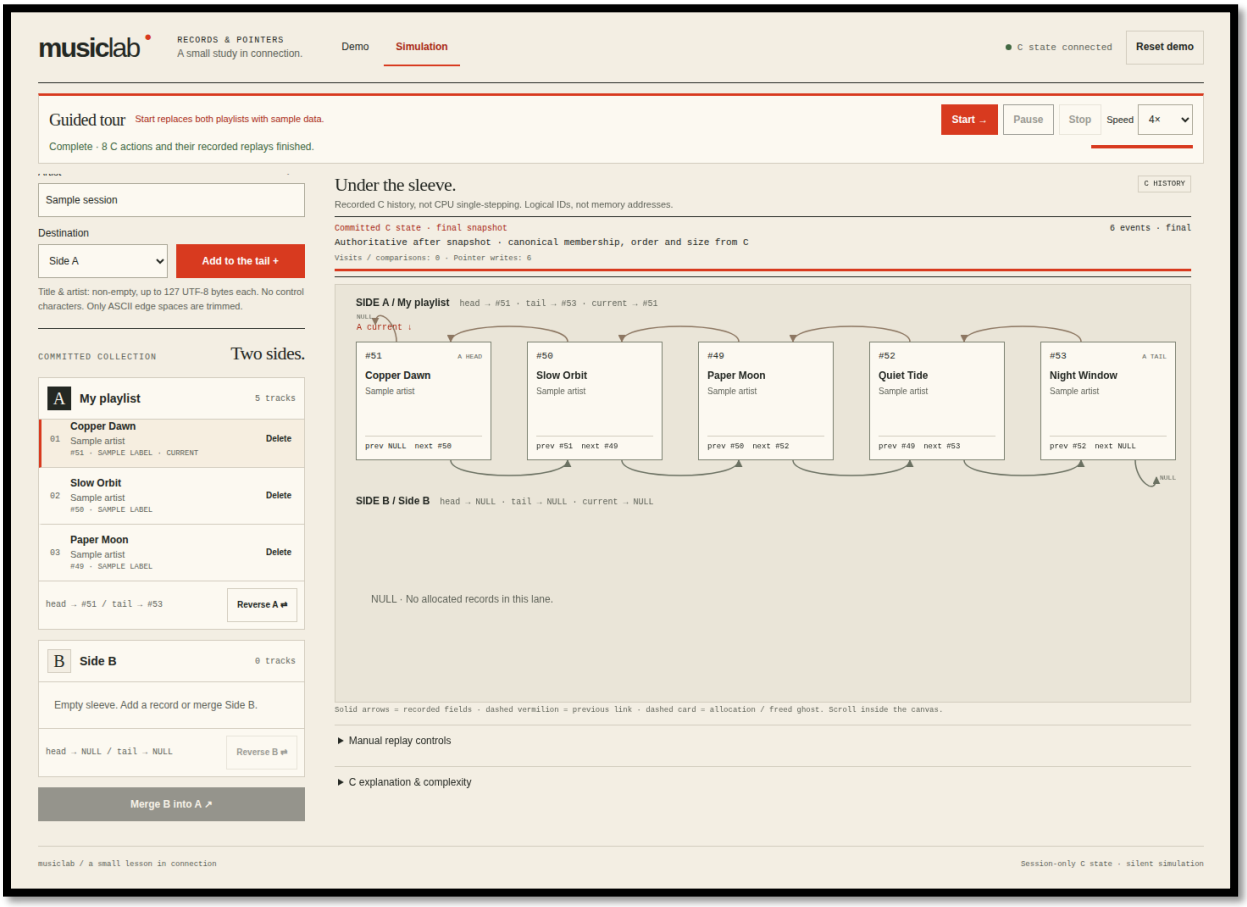



**Figure 5.** Complete screenshot after the guided run. Side A contains [51, 50, 49, 52, 53], with head = 51 and tail = 53. Side B is empty; its head, tail and current are NULL. [5]

| Visible check | Observed result |
|---|---|
| Reversed first playlist | Copper Dawn, Slow Orbit and Paper Moon appear in reversed order. |
| Joined endpoints | Node 49.next points to 52; node 52.prev points back to 49. |
| Source cleared | Side B shows NULL head, tail and current, with no allocated records. |

The tour adds node 54 and later deletes it while selected, leaving node 51 current. Reverse produces [51, 50, 49]; Merge then attaches [52, 53]. The final five IDs are existing records, and Side B is empty.

The browser test checks the eight actions, recorded frames and final C state shown here. [5]

The important result is the agreement between the final order and the link endpoints. A visual order alone would not reveal every incorrect backward link, so the screenshot is supported by the API states and invariant checks described in Section 7. [4, 7]

# 9. Analysis of Time and Space Complexity

Let n be the number of nodes in the relevant playlist and N the total across both playlists. Metadata fields have a fixed maximum size. The table describes the C list operation, excluding snapshots, JSON conversion and drawing. The configured limit is 200 songs; n and N describe how work grows as input size varies within that limit. [1, 2]

| Operation | Best | Average | Worst | Reason |
|---|---|---|---|---|
| Append | O(1) | O(1) | O(1) | Tail is stored; only endpoint links change. |
| Next / Previous | O(1) | O(1) | O(1) | Follow one link from current. |
| Search / Select by ID | O(1) | O(n) | O(n) | Traverse from head to the matching ID. |
| Delete by ID | O(1) | O(n) | O(n) | Search, then O(1) unlink and free. |
| Reverse | O(n) | O(n) | O(n) | Exchange prev/next for every node. |
| Concatenate | O(1) | O(1) | O(1) | Join endpoints and transfer ownership. |
| Display / Clear | O(n) | O(n) | O(n) | Visit or free every node. |

Searching for the head takes one comparison; searching for the tail takes n. If each position is equally likely, a successful search averages (n + 1) / 2 visits. A missing ID requires the full traversal. Deletion by ID includes this search, even though unlinking the located node is constant-time. Empty-list cases take constant time.

### Why constant-time concatenation is possible

T5 joins a two-node B to a three-node A by connecting A.tail with B.head and updating the stored endpoints and counters. The same set of assignments would join longer lists because their endpoints are already known. A sorted merge would need to compare records and is outside this implementation. [2, 7]

### Space requirements

Each song adds one fixed-size node, so storing N songs needs O(N) space. Beyond those nodes, the core algorithms use O(1) auxiliary space. Reverse uses temporary pointers to work through the existing list rather than allocating another list. The title and artist arrays have fixed capacities. [2]

The surrounding application does more work than the pointer operation alone. It saves the earlier state, records events and writes both playlists into the JSON response. This adds O(N) storage and O(N) request/response work even when a core operation is O(1). Drawing and replaying that response add browser work. [3]

### Reading the complexity results

These bounds describe growth as the list gets longer, not measured execution times. An O(1) operation can still perform several assignments. Conversely, a small list may make an O(n) search feel immediate. The 200-song limit bounds this application’s workload but does not change the traversal pattern of its algorithms. [2, 3]

# 10. Challenges and Learning Outcomes

## Keeping deletion safe

Deletion has two responsibilities: repair the list and keep a valid selection. In T3, removing selected node 2 moves current to node 3. When the last remaining node is removed in T8, all three playlist pointers become NULL. These cases explain why updating the neighbours alone is insufficient. [2, 7]

## Preserving traversal during reversal

Reverse changes the same links that traversal normally follows. Saving node.next first keeps access to the unprocessed part of the list. The dry run in Section 5 makes the assignment order visible: save the route, exchange the links, then move along the saved route. [2]

## Transferring ownership during concatenation

A join must define which playlist owns the transferred nodes. Musiclab gives them to A and clears B. T11 checks an empty destination, while T12 checks an empty source and rejects joining a list to itself. These cases protect against two lists retaining ownership of the same nodes. [2, 7]

## Making pointer changes understandable

The Simulation view records enough detail to explain an operation one assignment at a time. Its intermediate frames can differ from the final valid list, so the interface labels the recorded history separately. This distinction matters when explaining a temporarily disconnected link during the viva. [3]

**Learning outcomes.** The worked examples connect calloc and free with node lifetime, show why both link directions must agree, and distinguish the cost of finding a song from removing it. The empty-list tests also show why boundary cases need explicit handling.

**Future improvements.** The first priority is to reproduce and fix the replay keyboard-focus issue, then repeat the browser tests. Saving and loading metadata would make playlists persist across runs. A later ID index could shorten searches, but every insertion and deletion would also need to update that index.

## Connecting testing with pointer reasoning

A useful lesson is that a correct song order does not prove the whole structure is correct. Checking backward traversal, node count and current membership exposes errors that the player view could hide. The recorded event sequence also helps explain why a temporary replay frame differs from the completed list. These checks connect debugging with the invariants introduced in Section 4. [3, 4]

A future change should be tested against these same properties. For example, adding persistent storage would require rebuilding valid links from saved metadata rather than writing process-specific memory addresses to a file. The current implementation provides a base for exploring that extension without claiming it is already available.

# 11. Conclusion

Musiclab demonstrates a doubly linked list through operations that can be followed from a button click to C pointer assignments. The twelve report scenarios passed, including deletion to an empty list and rejected input. Stored endpoints support constant-time append and concatenation; ID searches and reversal require traversal. The report separates these costs from the work of recording and displaying results. The remaining limitations are temporary playlist storage and audio associations, plus the reported replay-focus issue. Within that scope, the code, worked examples and execution evidence support the project's second-year DSA objectives. [2, 7, 8]

## What the implementation establishes

The node structure is suitable for a playlist whose common operations involve adjacent navigation and local changes. Next and Previous use the two neighbour links directly. Tail makes appending straightforward, and the stored endpoints allow two lists to be joined without copying the song records. At the same time, looking up an arbitrary ID still requires traversal, so the design has a clear and explainable trade-off. [1, 2]

The tests show how these operations behave beyond the normal case. Removing the only song leaves an empty list, merging into an empty destination transfers the selected node, and invalid input leaves the state unchanged. The worked examples explain why those results follow from the assignments in the C source. [2, 7]

## Overall outcome and next step

The project meets its main DSA learning objectives by linking implementation, analysis and observable output. Its value for the viva is that each visible operation can be traced to a small set of functions and justified in terms of correctness and cost. The next practical step is to resolve the reported keyboard-focus behaviour and repeat browser validation. Persistence can then be considered as a separate extension. [5, 8]

# 12. References

[1] Pat Morin. Open Data Structures, Section 3.2, A Doubly-Linked List. Consulted 14 October 2026.

Used for linked-list operations and traversal costs. Its illustrated implementation uses a dummy node; Musiclab uses NULL endpoints.

[2] Musiclab source: src/playlist.h and src/playlist.c. Commit f162a82a.

[3] Project README, API.md and SIMULATION.md. Setup, behaviour and replay contract.

[4] tests/native.c, tests/trace.c, tests/api.mjs and web/check.cjs. Validation code.

[5] GitHub Actions run 38446276615 and tests/browser.cjs. Browser logs and screenshots, 13 October 2026 UTC.

[6] DSA CA2 Experiential Learning Report Template and Sample. Course-provided report instructions.

[7] Report verification records, 14 October 2026. make check log, actual API states, test script and retrieved CI evidence, included in the companion package.

[8] Rajveer Vadnal. Musiclab CA-2 DSA Report V1. Supplied earlier report, Section 7: keyboard-focus observation during browser replay.

## How the sources support this report

Reference [1] supports the discussion of linked-list operations and traversal. Its example uses a dummy node, so the report separately explains Musiclab's NULL endpoints. References [2] and [3] establish the implemented data fields, control flow and application limits. They are also the basis for distinguishing C list operations from browser playback and replay.

References [4], [5] and [7] support different forms of verification: test code, an archived browser run and the captured report scenarios. Their roles are kept separate because a screenshot, a test definition and a successful execution log establish different things. Reference [6] defines the report requirements, while [8] preserves the earlier interface observation.

Repository links identify revision f162a82a. This makes the implementation described here reproducible even if the main branch changes later. The companion archive provides the same source files and evidence records for offline inspection.

# Appendix A Source Code and Companion Package

The following pages reproduce src/playlist.h and src/playlist.c from the project repository at revision f162a82a. [2] The companion archive contains all 23 unchanged repository files, including the API, server, interface and tests, with instructions for building the complete application.

Archive: Musiclab_Complete_Source_and_Evidence_f162a82a.zip

Repository revision: f162a82a908e941138d151dfa305804c50e83dfd

Extract the archive, open Musiclab-f162a82a and follow README.md. The Evidence folder contains the report test results, reproduction script, verification logs and original CI screenshots. Source files were checked against their Git blob hashes; a manifest records file hashes. Generated binaries and browser downloads are excluded.

## Using the source attachment

Begin with playlist.h to identify the fields in SongNode and Playlist. Then follow playlist_add, playlist_delete_node, playlist_reverse and playlist_merge in playlist.c. Compare their assignments with the worked examples in Section 5. The API and browser files in the archive show how those functions are reached from the interface. [2, 3]

The printed code is preserved from the repository. Its observer calls record real operations for the Simulation view; they do not replace the list logic. When explaining the code, identify the list update first, then explain what the associated event allows the browser to display.

## Suggested demonstration sequence

| Time | Demonstration focus |
|---|---|
| 0–2 minutes | Explain the node model and navigate between songs. |
| 2–4 minutes | Append a song and delete a selected node; explain repaired links. |
| 4–6 minutes | Reverse the playlist and trace savedNext. |
| 6–8 minutes | Concatenate Side B; explain retained IDs and ownership. |
| 8–10 minutes | Demonstrate a boundary case and explain complexity and cleanup. |

# A1 Core Data Structures

Complete source listing | src/playlist.h | lines 1–56 | f162a82a

SongNode owns a song's metadata and links. Playlist stores its endpoints, selection, size and optional event observer. [2]

```
#ifndef PLAYLIST_H
#define PLAYLIST_H
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#define SONG_TEXT_BYTES 127
#define SONG_LIMIT 200
typedef struct SongNode {
    uint32_t id;
    char title[SONG_TEXT_BYTES + 1];
    char artist[SONG_TEXT_BYTES + 1];
    struct SongNode *prev, *next;
} SongNode;
typedef enum { PLAYLIST_ALLOCATE, PLAYLIST_VISIT, PLAYLIST_POINTER, PLAYLIST_FREE }
    PlaylistEventKind;
typedef enum { POINTER_PREV, POINTER_NEXT, POINTER_HEAD, POINTER_TAIL, POINTER_CURRENT }
    PlaylistPointer;
/* Events own IDs/metadata, never node addresses (including after a free). */
typedef struct {
    PlaylistEventKind kind;
    PlaylistPointer field;
    uint32_t id, from, to; /* zero denotes NULL */
    char title[SONG_TEXT_BYTES + 1], artist[SONG_TEXT_BYTES + 1];
} PlaylistEvent;
typedef struct Playlist Playlist;
/* Synchronous, read-only callback; do not mutate/reenter the list. */
typedef void (*PlaylistObserver)(void *context, const Playlist *list,
                                 const PlaylistEvent *event);
struct Playlist {
    SongNode *head, *tail, *current;
    size_t size;
    PlaylistObserver observer; /* optional, NULL by default */
    void *observer_context;
};
/* Validate UTF-8, trim ASCII spaces, reject control code points; never truncate. */
bool song_text(char out[SONG_TEXT_BYTES + 1], const char *input);
/* Caller supplies a fresh process-wide ID; nodes are owned by their playlist. */
SongNode *playlist_add(Playlist *list, uint32_t id, const char *title,
                       const char *artist);
SongNode *playlist_find(const Playlist *list, uint32_t id);
bool playlist_delete(Playlist *list, uint32_t id);
bool playlist_select(Playlist *list, uint32_t id);
/* Node must belong to list; reuse a successful find without another traversal. */
void playlist_delete_node(Playlist *list, SongNode *node);
void playlist_select_node(Playlist *list, SongNode *node);
bool playlist_move(Playlist *list, bool forward);
void playlist_reverse(Playlist *list);
bool playlist_merge(Playlist *destination, Playlist *source);
void playlist_clear(Playlist *list);
#endif
```

# A2 Event Recording and Metadata Validation

Complete source listing | src/playlist.c | lines 1–61 | f162a82a

Events identify nodes by ID; song_text checks the metadata before insertion. [2]

```
#include "playlist.h"
#include <stdlib.h>
#include <string.h>
static uint32_t node_id(const SongNode *node) { return node ? node->id : 0; }
static void emit(Playlist *list, PlaylistEventKind kind, const SongNode *node)
{
    PlaylistEvent event = {.kind = kind, .id = node_id(node)};
    if (kind == PLAYLIST_ALLOCATE) {
        strcpy(event.title, node->title);
        strcpy(event.artist, node->artist);
    }
    list->observer(list->observer_context, list, &event);
}
/* Disabled tracing costs only the branch; record the actual assignment, even a no-op. */
#define ASSIGN(list, owner, field_name, slot, target) do { \
    if ((list)->observer) { \
        PlaylistEvent event = {.kind = PLAYLIST_POINTER, .field = (field_name), \
            .id = node_id(owner), .from = node_id(slot)}; \
        (slot) = (target); \
        event.to = node_id(slot); \
        (list)->observer((list)->observer_context, (list), &event); \
    } else { (slot) = (target); } \
} while (0)
bool song_text(char out[SONG_TEXT_BYTES + 1], const char *input)
{
    const unsigned char *start = (const unsigned char *)input;
    while (*start == ' ') start++;
    size_t length = strlen((const char *)start);
    while (length && start[length - 1] == ' ') length--;
    if (!length || length > SONG_TEXT_BYTES) return false;
    for (size_t i = 0; i < length;) {
        uint32_t cp = start[i++];
        unsigned extra = 0;
        uint32_t minimum = 0;
        if (cp >= 0xc2 && cp <= 0xdf) {
            extra = 1; minimum = 0x80; cp &= 0x1f;
        } else if (cp >= 0xe0 && cp <= 0xef) {
            extra = 2; minimum = 0x800; cp &= 0x0f;
        } else if (cp >= 0xf0 && cp <= 0xf4) {
            extra = 3; minimum = 0x10000; cp &= 7;
        } else if (cp >= 0x80) {
            return false;
        }
        if (extra > length - i) return false;
        for (unsigned j = 0; j < extra; j++) {
            if ((start[i] & 0xc0) != 0x80) return false;
            cp = (cp << 6) | (start[i++] & 0x3f);
        }
        if (cp < minimum || cp > 0x10ffff || (cp >= 0xd800 && cp <= 0xdfff)
            || cp < 0x20 || (cp >= 0x7f && cp <= 0x9f)) return false;
    }
    memcpy(out, start, length);
    out[length] = '\0';
    return true;
}
```

# A3 Append Search and Deletion

Complete source listing | src/playlist.c | lines 62–115 | f162a82a

Compare deletion by ID with deletion of an already located node. The latter performs the neighbour and endpoint repairs. [2]

```
SongNode *playlist_add(Playlist *list, uint32_t id, const char *title,
                       const char *artist)
{
    char clean_title[SONG_TEXT_BYTES + 1], clean_artist[SONG_TEXT_BYTES + 1];
    if (!id || list->size >= SONG_LIMIT || !song_text(clean_title, title)
        || !song_text(clean_artist, artist)) return NULL;
    SongNode *node = calloc(1, sizeof(*node));
    if (!node) return NULL;
    node->id = id;
    strcpy(node->title, clean_title);
    strcpy(node->artist, clean_artist);
    if (list->observer) emit(list, PLAYLIST_ALLOCATE, node);
    ASSIGN(list, node, POINTER_PREV, node->prev, list->tail);
    if (list->tail) ASSIGN(list, list->tail, POINTER_NEXT, list->tail->next, node);
    else ASSIGN(list, NULL, POINTER_HEAD, list->head, node);
    ASSIGN(list, NULL, POINTER_TAIL, list->tail, node);
    if (!list->current) ASSIGN(list, NULL, POINTER_CURRENT, list->current, node);
    list->size++;
    return node;
}
SongNode *playlist_find(const Playlist *list, uint32_t id)
{
    for (SongNode *node = list->head; node; node = node->next) {
        if (list->observer) {
            PlaylistEvent event = {.kind = PLAYLIST_VISIT, .id = node->id};
            list->observer(list->observer_context, list, &event);
        }
        if (node->id == id) return node;
    }
    return NULL;
}
bool playlist_delete(Playlist *list, uint32_t id)
{
    SongNode *node = playlist_find(list, id);
    if (!node) return false;
    playlist_delete_node(list, node);
    return true;
}
void playlist_delete_node(Playlist *list, SongNode *node)
{
    if (list->current == node)
        ASSIGN(list, NULL, POINTER_CURRENT, list->current, node->next ? node->next :
            node->prev);
    if (node->prev) ASSIGN(list, node->prev, POINTER_NEXT, node->prev->next, node->next);
    else ASSIGN(list, NULL, POINTER_HEAD, list->head, node->next);
    if (node->next) ASSIGN(list, node->next, POINTER_PREV, node->next->prev, node->prev);
    else ASSIGN(list, NULL, POINTER_TAIL, list->tail, node->prev);
    list->size--;
    if (list->observer) emit(list, PLAYLIST_FREE, node);
    free(node);
}
```

# A4 Selection Navigation and Reversal

Complete source listing | src/playlist.c | lines 116–151 | f162a82a

Selection changes current. Reversal exchanges the neighbour links and then the two endpoints, while current remains on its node. [2]

```
bool playlist_select(Playlist *list, uint32_t id)
{
    SongNode *node = playlist_find(list, id);
    if (!node) return false;
    playlist_select_node(list, node);
    return true;
}
void playlist_select_node(Playlist *list, SongNode *node)
{
    ASSIGN(list, NULL, POINTER_CURRENT, list->current, node);
}
bool playlist_move(Playlist *list, bool forward)
{
    if (!list->current) return false;
    SongNode *next = forward ? list->current->next : list->current->prev;
    if (!next) return false;
    ASSIGN(list, NULL, POINTER_CURRENT, list->current, next);
    return true;
}
void playlist_reverse(Playlist *list)
{
    SongNode *node = list->head;
    while (node) {
        SongNode *next = node->next;
        ASSIGN(list, node, POINTER_NEXT, node->next, node->prev);
        ASSIGN(list, node, POINTER_PREV, node->prev, next);
        node = next;
    }
    SongNode *head = list->head;
    ASSIGN(list, NULL, POINTER_HEAD, list->head, list->tail);
    ASSIGN(list, NULL, POINTER_TAIL, list->tail, head);
}
```

# A5 Concatenation and Memory Cleanup

Complete source listing | src/playlist.c | lines 152–186 | f162a82a

Concatenation transfers ownership and clears the source. Cleanup saves each next pointer before releasing the current node. [2]

```
bool playlist_merge(Playlist *destination, Playlist *source)
{
    if (destination == source || destination->size + source->size > SONG_LIMIT)
        return false;
    if (!source->head) return true;
    if (destination->tail) {
        ASSIGN(destination, destination->tail, POINTER_NEXT, destination->tail->next,
            source->head);
        ASSIGN(source, source->head, POINTER_PREV, source->head->prev, destination->tail);
    } else {
        ASSIGN(destination, NULL, POINTER_HEAD, destination->head, source->head);
        ASSIGN(destination, NULL, POINTER_CURRENT, destination->current, source->current);
    }
    ASSIGN(destination, NULL, POINTER_TAIL, destination->tail, source->tail);
    destination->size += source->size;
    ASSIGN(source, NULL, POINTER_HEAD, source->head, NULL);
    ASSIGN(source, NULL, POINTER_TAIL, source->tail, NULL);
    ASSIGN(source, NULL, POINTER_CURRENT, source->current, NULL);
    source->size = 0;
    return true;
}
void playlist_clear(Playlist *list)
{
    SongNode *node = list->head;
    ASSIGN(list, NULL, POINTER_HEAD, list->head, NULL);
    ASSIGN(list, NULL, POINTER_TAIL, list->tail, NULL);
    ASSIGN(list, NULL, POINTER_CURRENT, list->current, NULL);
    while (node) {
        SongNode *next = node->next;
        if (list->observer) emit(list, PLAYLIST_FREE, node);
        free(node);
        node = next;
    }
    list->size = 0;
}
```

# AI CONTENT GUARDIAN

## AUTOMATED FORENSIC REPORT

Date: 10/20/2026
File: **DSA Report Finalised.docx**
Verdict ID: #696263

### DOCUMENT SCORE BREAKDOWN

| TEXT-ONLY AI SCORE | IMAGE-ONLY AI SCORE | COMBINED VERDICT |
|---|---|---|
| **28.2%** | **20.1%** | **25.8%** |
| Mostly Human (Minor enhancements) | Avg of 13 images | LIKELY HUMAN AUTHORED |
| | | (28.2% × 70%) + (20.1% × 30%) |

### ANALYSIS CONTEXT

| | |
|---|---|
| Tool Version | V2.1.0-FORENSIC |
| Analysis Mode | DOCUMENT |
| Word Count | 5793 Words |

### TEXT FORENSICS

| | |
|---|---|
| Entropy | 4.59 |
| Perplexity | 24.02 |
| Burstiness | 0.80 |
| Lexical TTR | 0.58 |

### EXECUTIVE SUMMARY

Analyzed 5793 words and 13 images.

# FORENSICS DATABASE - EMBEDDINGS CHUNK 1

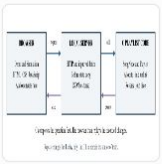


**image3.jpg**
Spectral Art: **8.79** Perplexity: **2.62**

REAL / HUMAN

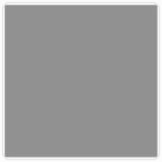


**image4.png**
Spectral Art: **0.00** Perplexity: **0.00**

AI



**image5.jpg**
Spectral Art: **8.97** Perplexity: **2.32**

REAL / HUMAN



**image7.jpeg**
Spectral Art: **8.97** Perplexity: **2.32**

REAL / HUMAN

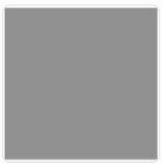


**image10.png**
Spectral Art: **0.00** Perplexity: **0.00**

AI



**image11.jpeg**
Spectral Art: **9.06** Perplexity: **3.03**

REAL / HUMAN



**image12.jpg**
Spectral Art: **9.12** Perplexity: **3.07**

REAL / HUMAN



**image13.jpeg**
Spectral Art: **9.12** Perplexity: **3.07**

REAL / HUMAN

# FORENSICS DATABASE - EMBEDDINGS CHUNK 2



**image9.jpg**
Spectral Art: **9.06** Perplexity: **3.03**

REAL / HUMAN



**image1.jpg**
Spectral Art: **8.00** Perplexity: **4.30**

REAL / HUMAN

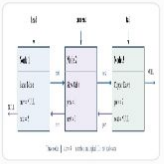


**image2.jpg**
Spectral Art: **8.43** Perplexity: **2.60**

REAL / HUMAN



**image6.png**
Spectral Art: **0.00** Perplexity: **0.00**

AI



**image8.png**
Spectral Art: **0.00** Perplexity: **0.00**

AI